

Chapter 7: Java Exception Handling

- An exception is an abnormal condition that arises in a code sequence at run time.
- In other words, an exception is a run-time error. In computer languages that do not support exception handling, errors must be checked and handled manually—typically through the use of error codes, and so on.
- This approach is as cumbersome as it is troublesome.
- Java's exception handling avoids these problems and, in the process, brings run-time error management into the object oriented world.

6.1. Error and Exception

Both Error and Exception are derived from *java.lang.Throwable* in Java but main difference between Error and Exception is kind of error they represent. *java.lang.Error* represent errors which are generally can not be handled and usually refer catastrophic failure e.g. *running out of System resources*, some examples of Error in Java are *java.lang.OutOfMemoryError* or *Java.lang.NoClassDefFoundError* and *java.lang.UnsupportedClassVersionError*. On the other hand *java.lang.Exception* represent errors which can be catch and dealt e.g. *IOException* which comes while performing I/O operations i.e. reading files and directories.

6.2. Exception-Handling Fundamentals

A Java *exception* is an object that describes an exceptional (that is, error) condition that has occurred in a piece of code. When an exceptional condition arises, an *object* representing that exception is created and thrown in the method that caused the error. That method may choose to handle the exception itself, or pass it on. Either way, at some point, the exception is caught and processed. Exceptions can be generated by the *Java run-time* system, or they can be manually generated by our code. Exceptions thrown by Java relate to fundamental errors that violate the rules of the Java language or the constraints of the Java execution environment.

Java exception handling is managed via five keywords: ***try, catch, throw, throws, and finally***. Briefly, here is how they work.

- Program statements that we want to monitor for exceptions are contained within a try block.
- If an exception occurs within the try block, it is thrown.
- Our code can catch this exception (using catch) and handle it in some rational manner.
- System-generated exceptions are automatically thrown by the Java run-time system.
- To manually throw an exception, use the keyword throw.
- Any exception that is thrown out of a method must be specified as such by a throws clause. Any code that absolutely must be executed after a try block completes is put in a finally block.

This is the general form of an exception-handling block:

```
try {  
    // block of code to monitor for errors  
}  
catch (ExceptionType1 exOb) {  
    // exception handler for ExceptionType1  
}  
catch (ExceptionType2 exOb) {  
    // exception handler for ExceptionType2  
}  
finally {  
    // block of code to be executed after try block ends  
}
```

Here, *ExceptionType* is the type of exception that has occurred.

6.3. Exception Types

All exception types are subclasses of the built-in class **Throwable**. Thus, Throwable is at the top of the exception class hierarchy. Immediately below Throwable are two subclasses that partition exceptions into two distinct branches. One branch is headed by *Exception*. This class is used for exceptional conditions that user programs should catch. This is also the class that we will subclass to create our own custom exception types. There is an important subclass of Exception, called *RuntimeException*. Exceptions of this type are automatically defined for the programs that we write and include things such as division by zero and invalid array indexing.

The other branch is topped by *Error*, which defines exceptions that are not expected to be caught under normal circumstances by our program. Exceptions of type *Error* are used by the Java run-time system to indicate errors having to do with the run-time environment, itself. Stack overflow is an example of such an error.

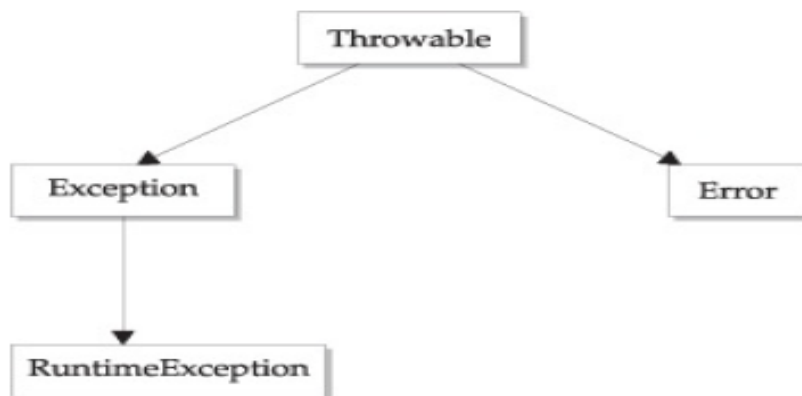


Fig: Top Level Exception Hierarchy

6.4. Uncaught Exceptions

It is useful to see what happens when we don't handle them. This small program includes an expression that intentionally causes a divide-by-zero error:

```
class Exc0 {  
public static void main(String args[]) {  
int d = 0;  
int a = 42 / d;  
}  
}
```

When the Java run-time system detects the attempt to divide by zero, it constructs a new exception object and then throws this exception. This causes the execution of Exc0 to stop, because once an exception has been thrown, it must be caught by an exception handler and dealt with immediately. In this example, we haven't supplied any exception handlers of our own, so the default handler provided by the Java run-time system catches the exception. The default handler will ultimately process any exception that is not caught by our program. The default handler displays a string describing the exception, prints a stack trace from the point at which the exception occurred, and terminates the program.

Here is the exception generated when this example is executed:

```
java.lang.ArithmeticException: / by zero  
at Exc0.main(Exc0.java:4)
```

6.5. Using *try* and *catch*

Although the default exception handler provided by the Java run-time system is useful for debugging, we will usually want to handle an exception our self. Doing so provides two benefits. First, *it allows us to fix the error*. Second, *it prevents the program from automatically terminating*. To guard against and

handle a run-time error, simply enclose the code that we want to monitor inside a try block. Immediately following the try block, include a catch clause that specifies the exception type that we wish to catch. Following block of code shows that the *ArithmeticException* generated by the division-by-zero error:

```
class Exc2 {
    public static void main(String args[]) {
        int d, a;
        try { // monitor a block of code.
            d = 0;
            a = 42 / d;
            System.out.println("This will not be printed.");
        } catch (ArithmeticException e) { // catch divide-by-zero error
            System.out.println("Division by zero.");
        }
        System.out.println("After catch statement.");
    }
}
```

This program generates the following output:

Division by zero.

After catch statement.

Once the catch statement has executed, program control continues with the next line in the program following the entire try/catch mechanism.

6.6. Multiple *catch* Clauses

In some cases, more than one exception could be raised by a single piece of code. To handle this type of situation, we can specify two or more catch clauses, each catching a different type of exception. When an exception is thrown, each catch statement is inspected in order, and the first one whose type matches that of the exception is executed. After one catch statement executes, the others are bypassed, and execution continues after the try/catch block. The following example traps two different exception types:

```
class MultiCatch {
    public static void main(String args[]) {
        try {
            int a = args.length;
            System.out.println("a = " + a);
            int b = 42 / a;
            int c[] = { 1 };
            c[42] = 99;
        } catch (ArithmeticException e) {
            System.out.println("Divide by 0: " + e);
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Array index oob: " + e);
        }
        System.out.println("After try/catch blocks.");
    }
}
```

This program will cause a division-by-zero exception if it is started with no commandline arguments, since *a* will equal zero. It will survive the division if we provide a command-line argument, setting *a* to something larger than zero. But it will cause an *ArrayIndexOutOfBoundsException*, since the int array *c* has a length of 1, yet the program attempts to assign a value to *c*[42].

Here is the output generated by running it both ways:

C:\>java MultiCatch

```
a = 0
Divide by 0: java.lang.ArithmeticException: / by zero
After try/catch blocks.
C:\>java MultiCatch TestArg
a = 1
Array index oob: java.lang.ArrayIndexOutOfBoundsException:42
After try/catch blocks.
```

When using multiple catch statements, it is important to remember that exception subclasses must come before any of their superclasses. This is because a catch statement that uses a superclass will catch exceptions of that type plus any of its subclasses. Thus, a subclass would never be reached if it came after its superclass. *Further, in Java, unreachable code is an error.* For example, consider the following program:

```
class SuperSubCatch {
    public static void main(String args[]) {
        try {
            int a = 0;
            int b = 42 / a;
        } catch(Exception e) {
            System.out.println("Generic Exception catch.");
        }
        /* This catch is never reached because
        ArithmeticException is a subclass of Exception. */
        catch(ArithmeticException e) { // ERROR - unreachable
            System.out.println("This is never reached.");
        }
    }
}
```

If we try to compile this program, we will receive an error message stating that the *second catch statement is unreachable because the exception has already been caught*. Since *ArithmeticException* is a subclass of *Exception*, the first catch statement will handle all *Exception*-based errors, including *ArithmeticException*. This means that the second catch statement will never execute. To fix the problem, reverse the order of the catch statements.

6.7. Nested try Statements

The *try* statement can be nested. That is, a *try* statement can be inside the block of another *try*. Each time a *try* statement is entered, the context of that exception is pushed on the stack. If an inner *try* statement does not have a catch handler for a particular exception, the stack is unwound and the next *try* statement's catch handlers are inspected for a match. This continues until one of the catch statements succeeds, or until all of the nested *try* statements are exhausted. If no catch statement matches, then the Java run-time system will handle the exception. Here is an example that uses nested *try* statements:

```
// An example of nested try statements.
class NestTry {
    public static void main(String args[]) {
        try {
            int a = args.length;
            /* If no command-line args are present, the following statement will generate a divide-by-zero
            exception. */
            int b = 42 / a;
            System.out.println("a = " + a);
            try { // nested try block
                /* If one command-line arg is used, then a divide-by-zero exception will be generated by the following
                code. */
```

```

        if(a==1) a = a/(a-a); // division by zero
/* If two command-line args are used, then generate an out-of-bounds exception. */
        if(a==2) {
            int c[] = { 1 };
            c[42] = 99; // generate an out-of-bounds exception
        }
        catch(ArrayIndexOutOfBoundsException e) {
            System.out.println("Array index out-of-bounds: "
+ e);
        }
        catch(ArithmeticException e) {
            System.out.println("Divide by 0: " + e);
        }
    }
}

```

The program works as follows. When we execute the program with no command-line arguments, a divide-by-zero exception is generated by the outer try block. Execution of the program with one command-line argument generates a divide-by-zero exception from within the nested try block. Since the inner block does not catch this exception, it is passed on to the outer try block, where it is handled. If we execute the program with two command-line arguments, an array boundary exception is generated from within the inner try block.

6.8. throw

However, it is possible for our program to throw an exception explicitly, using the **throw** statement. The general form of throw is shown here:

throw ThrowableInstance;

Here, *ThrowableInstance* must be an object of type *Throwable* or a subclass of *Throwable*. Primitive types, such as *int* or *char*, as well as *non-Throwable* classes, such as *String* and *Object*, cannot be used as exceptions. There are two ways that we can obtain a *Throwable* object: using a parameter in a catch clause, or creating one with the new operator.

The flow of execution stops immediately after the throw statement; any subsequent statements are not executed. The nearest enclosing try block is inspected to see if it has a catch statement that matches the type of exception. If it does find a match, control is transferred to that statement. If not, then the next enclosing try statement is inspected, and so on. If no matching catch is found, then the default exception handler halts the program and prints the stack trace.

```

// Demonstrate throw.
class ThrowDemo {
    static void demoproc() {
        try {
            throw new NullPointerException("demo");
        } catch(NullPointerException e) {
            System.out.println("Caught inside demoproc.");
            throw e; // rethrow the exception
        }
    }

    public static void main(String args[]) {
        try {
            demoproc();
        } catch(NullPointerException e) {
            System.out.println("Recaught: " + e);
        }
    }
}

```

```
}
```

This program gets two chances to deal with the same error. First, `main( )` sets up an exception context and then calls `demoproc( )`. The `demoproc( )` method then sets up another *exceptionhandling* context and immediately throws a new instance of `NullPointerException`, which is caught on the next line. The exception is then rethrown. Here is the resulting output:

Caught inside demoproc.

Recaught: java.lang.NullPointerException: demo

6.9. throws

If a method is capable of causing an exception that it does not handle, it must specify this behavior so that callers of the method can guard themselves against that exception. You do this by including a `throws` clause in the method's declaration. A `throws` clause lists the types of exceptions that a method might throw. This is necessary for all exceptions, except those of type **Error or RuntimeException**, or any of their subclasses. All other exceptions that a method can throw must be declared in the `throws` clause. If they are not, a compile-time error will result. This is the general form of a method declaration that includes a `throws` clause:

type method-name(parameter-list) throws exception-list

```
{  
// body of method  
}
```

Here, `exception-list` is a comma-separated list of the exceptions that a method can throw.

```
class ThrowsDemo {  
    static void throwOne() throws IllegalAccessException {  
        System.out.println("Inside throwOne.");  
        throw new IllegalAccessException("demo");  
    }  
    public static void main(String args[]) {  
        try {  
            throwOne();  
        } catch (IllegalAccessException e) {  
            System.out.println("Caught " + e);  
        }  
    }  
}
```

Here is the output generated by running this example program:

inside throwOne

caught java.lang.IllegalAccessException: demo

6.10. finally

When exceptions are thrown, execution in a method takes a rather abrupt, nonlinear path that alters the normal flow through the method. Depending upon how the method is coded, it is even possible for an exception to cause the method to return prematurely. This could be a problem in some methods. For example, if a method opens a file upon entry and closes it upon exit, then we will not want the code that closes the file to be bypassed by the exception-handling mechanism. The `finally` keyword is designed to address this contingency.

finally creates a block of code that will be executed after a *try/catch* block has completed and before the code following the *try/catch* block. The *finally* block will execute whether or not an exception is thrown. If an exception is thrown, the *finally* block will execute even if no *catch* statement matches the exception. Any time a method is about to return to the caller from inside a *try/catch* block, via an uncaught exception or an explicit return statement, the *finally* clause is also executed just before the method returns. This can be useful for closing file handles and freeing up any other resources that might have been allocated at the beginning of a method with the intent of disposing of them before returning.

The *finally* clause is optional. However, each try statement requires at least one catch or a finally clause. Here is an example program that shows three methods that exit in various ways, none without executing their finally clauses:

// Demonstrate finally.

```
class FinallyDemo {
    // Through an exception out of the method.
    static void procA() {
        try {
            System.out.println("inside procA");
            throw new RuntimeException("demo");
        } finally {
            System.out.println("procA's finally");
        }
    }
    // Return from within a try block.
    static void procB() {
        try {
            System.out.println("inside procB");
            return;
        } finally {
            System.out.println("procB's finally");
        }
    }
    // Execute a try block normally.
    static void procC() {
        try {
            System.out.println("inside procC");
        } finally {
        }
    }
    public static void main(String args[]) {
        try {
            procA();
        } catch (Exception e) {
            System.out.println("Exception caught");
        }
        procB();
        procC();
    }
}
```

In this example, `procA()` prematurely breaks out of the try by throwing an exception. The *finally* clause is executed on the way out. `procB()`'s try statement is exited via a return statement. The finally clause is executed before `procB()` returns. In `procC()`, the try statement executes normally, without error. However, the finally block is still executed.

Here is the output generated by the preceding program:

```
inside procA
procA's finally
Exception caught
inside procB
procB's finally
inside procC
procC's finally
```


6.11. Debugging

6.11.1. Loop Bug

The two most common kinds of loop errors are unintended *infinite loops* and *off-by-one errors*.

Infinite Loop: A while loop, do-while loop, or for loop does not terminate as long as the controlling Boolean expression evaluates to true. This Boolean expression normally contains a variable that will be changed by the loop body, and usually the value of this variable eventually is changed in a way that makes the Boolean expression false and therefore terminates the loop. However, if we make a mistake and write our program so that the Boolean expression is always true, then the loop will run forever. A loop that runs forever is called an **infinite loop**.

off-by-one error: The loop repeats the loop body *one too many or one too few times*. These sorts of errors can result from carelessness in designing a controlling Boolean expression. For example, if we use less-than when we should use less-than-or-equal, this can easily make our loop iterate the body the wrong number of times.

Use of `==` to test for equality in the controlling Boolean expression of a loop can often lead to an off-by-one error or an infinite loop. This sort of equality testing can work satisfactorily for integers and characters, but is not reliable for floating-point numbers. This is because the floating-point numbers are approximate quantities, and `==` tests for exact equality. The result of such a test is unpredictable. When comparing floating-point numbers, always use something involving less-than or greater-than, such as `<=`; do not use `==` or `!=`. Using `==` or `!=` to test floating-point numbers can produce an off-by-one error or an unintended infinite loop or even some other type of error. Even when using integer variables, it is best to avoid using `==` and `!=` and to instead use something involving less-than or greater-than.

6.11.2. Tracing Variables

One good way to discover errors in a loop or any kind of code is to trace some key variables. Tracing variables means watching the variables change value while the program is running. Most programs do not output each variable's value every time the variable changes, but being able to see all of these variable changes can help us to debug your program.

6.11.3. General Debugging Techniques

Tracing errors can sometimes be a difficult and time-consuming task. It is not uncommon to spend more time debugging a piece of code than it took to write the code in the first place. If we are having difficulties finding the source of our errors, then there are some general debugging techniques to consider. If the program is giving incorrect output values, then we should examine the source code, different test cases using a range of input and output values, and the logic behind the algorithm itself.

Determining the precise cause and location of a bug is one of the first steps in fixing the error. Examine the input and output behavior for different test cases to try to localize the error. A related technique is to trace variables to show what code the program is executing and what values are contained in key variables. We might also focus on code that has recently changed or code that has had errors before.

Finally, we can also try removing code. If we comment out blocks of code and the error still remains, then the culprit is in the uncommented code. The process can be repeated until the location of the error can be pinpointed. The `/*` and `*/` notation is particularly useful to comment out large blocks of code. After the error has been fixed, it is easy to remove the comments and reactivate the code.

The first mistakes we should look for are common errors that are easy to make. Examples of common errors include off-by-one errors, comparing floating-point types with `==`, adding extra semicolons that terminate a loop early, or using `==` to compare strings for equality.